URDU-INK MOBILE: A GAN-ENHANCED REAL-TIME HANDWRITING RECOGNITION KEYBOARD

[Place university logo as numl-logo.png]

Waqar Ahmad

Isna Jadoon

Supervised By

Dr. QuaratulAin Safdar

Submitted for the partial fulfillment of BS Computer Science degree to the Faculty of Engineering & CS

DEPARTMENT OF COMPUTER SCIENCE

NATIONAL UNIVERSITY OF MODERN LANGUAGES, ISLAMABAD

November, 2026

ABSTRACT

Urdu is the language of more than 230 million people across the world, yet slow and unnatural phonetic or fixed-layout keyboards are used to write it on digital devices like smartphones. Our project presents *Urdu-Ink Mobile*, a real-time handwriting recognition keyboard that users can use to write Urdu naturally with their finger or stylus and then convert the handwritten word into editable Unicode text. Preexisting academic systems only recognize isolated characters or ligatures, while commercial offerings like Gboard offer very limited support for Nastaliq style handwriting. This project extends the half-form recognition pipeline of Safdar et al. (2022), which uses pre-classification and wavelet-feature to achieve an accuracy of 96.1% to recognize full-form words.

The system is organized in multiple stages. Raw handwriting data in the form of XY coordinates, time and pressure values was collected using a flutter based canvas. The raw data was then passed through a jitter removal, downsampling, interpolation and normalization pre-processing pipeline to prepare the dataset for model training. A GAN network was trained to on the collected dataset to generate synthetic samples to extend the dataset. A Neural Network was then trained on both the organic and synthetic samples to recognize handwritten Urdu words. The models were developed using Python libraries PyTorch and Keras and deployed using TenosrFlow lite.

The system was tested using unit, integration, black-box and performance testing on Infinix Hot 11 Play with MediaTek Helio G36 processor with 4 GB of RAM running on Android 11 and .[device model, processor, RAM and Android/iOS version]. Recognition accuracy was measured using Character Error Rate and Word Error Rate. The tests showed a CER of [value]% and a WER of [value]% on [test set description], with an average end-to-end latency of [value] ms per word, all of which met [or did not meet] the targets set in the non-functional requirements.

The current system is limited by the size of the dataset. It can lose accuracy on very complex and long words. Expanding the dataset to include more writers and samples could be possible future works.

CERTIFICATE

Final Approval

Dated: _____

It is certified that the project report titled **'Urdu-Ink Mobile: A GAN-Enhanced Real-Time Handwriting Recognition Keyboard'** submitted by **Waqar Ahmad** and **Isna Jadoon** for the partial fulfillment of the requirement of **"Bachelor's Degree in Computer Science"** is approved.

COMMITTEE

[Name of Dean] Signature: _____
Acting Dean Engineering & CS:

Dr. Noman Malik Signature: _____
HoD Computer Science:

[Name of Coordinator] Signature: _____
Project Coordinator:

Dr. QuaratulAin Safdar Signature: _____
Supervisor:

DECLARATION

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group members:

Name	**Signature**
Waqar Ahmad	_____
Isna Jadoon	_____

PLAGIARISM CERTIFICATE

This is to certify that the project entitled "**Urdu-Ink Mobile: A GAN-Enhanced Real-Time Handwriting Recognition Keyboard**", which is being submitted herewith for the award of the "**Degree of Bachelor's in Computer Science**". This is the result of the original work by **Waqar Ahmad** and **Isna Jadoon** under my supervision and guidance. The work embodied in this project has not been done earlier for the basis of award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

Turnitin Originality Report
Processed on [dd-Mon-yyyy hh:mm PKT]
ID: [Turnitin ID]
Word Count: [count]

Similarity Index
[xx]%

Similarity by Source
Internet Sources: [xx]%
Publications: [xx]%
Student Papers: [xx]%

Date: [dd/mm/yyyy]

_____
Dr. QuaratulAin Safdar (Supervisor)

ACKNOWLEDGMENT

We express our sincere gratitude to our supervisor, Mam QuratUl Ain Safdar, for her guidance, patience and constructive feedback throughout this project. We also thank the volunteers who contributed their Urdu handwriting samples, and our families for their constant support.

TABLE OF CONTENTS

Chapter **Page**

CHAPTER 1: INTRODUCTION 1
1.1 Project Domain 2
1.2 Problem Identification 2
1.2.1 Proposed Solution 3
1.2.2 Objectives 3
1.2.3 Scope of the Project 3
1.3 Effectiveness / Usefulness of the System 3
1.4 Resource Requirement 4
1.4.1 Hardware Requirement 4
1.4.2 Software Requirement 4
1.4.3 Data Requirement 5
1.5 Report Organization 5

CHAPTER 2: BACKGROUND AND EXISTING SYSTEMS 6
2.1 Related Literature Review 7
2.2 Related Systems/Applications 9
2.3 Identified Problem from Existing Work 10
2.4 Selected Boundary for Proposed Solution 10

CHAPTER 3: SYSTEM REQUIREMENTS AND SPECIFICATIONS 12
3.1 System Specification 13
3.2 System Modules 13
3.2.1 Data Acquisition Module 13
3.2.2 Urdu Handwriting Corpus Module 14
3.2.3 Preprocessing Engine 14
3.2.4 RefinerGAN Module 14
3.2.5 Ligature Segmentation Module 14
3.2.6 Pre-Classifier Module 14
3.2.7 Hybrid Feature Extraction Module 15
3.2.8 Recognition Core 15
3.2.9 Language Model Layer 15
3.2.10 Real-Time Inference Engine 15
3.2.11 Evaluation and Analytics Module 15
3.3 Functional Requirements/Software Features 15
3.3.1 Stroke Capture (FR1) 16
3.3.2 Stroke Preprocessing (FR2) 16
3.3.3 Stroke Refinement (FR3) 16
3.3.4 Segmentation and Pre-classification (FR4) 16
3.3.5 Handwriting Recognition (FR5) 16

3.3.6 Language-Model Correction and Suggestions (FR6) 16
3.3.7 Unicode Insertion (FR7) 16
3.3.8 Text Correction and Editing (FR8) 16
3.3.9 Keyboard Management (FR9) 17
3.3.10 Performance Evaluation (FR10) 17
3.4 Non-Functional Requirements 17
3.4.1 Recognition Accuracy 17
3.4.2 Speed 17
3.4.3 Usability 17
3.4.4 Privacy 18
3.4.5 Size and Resource Use 18
3.4.6 Reliability and Robustness 18
3.4.7 Portability 18

CHAPTER 4: SYSTEM MODELING AND DESIGN 19
4.1 System Design and Analysis 20
4.2 Use Case Diagrams 20
4.3 Full Dress Use Case/Detailed Use Case 20
4.3.1 Write Urdu Handwriting – Full Dress Use Case 21
4.3.2 Select Suggested Word – Full Dress Use Case 22
4.4 Activity Diagram 22
4.5 Data Flow Diagram 23
4.6 System Sequence Diagram 24
4.7 Sequence Diagram 24
4.8 Design Class Diagram 25
4.9 Architectural Diagrams 26
4.9.1 Interface Design 27
4.9.2 Component Level Design 27
4.9.3 Deployment 28

CHAPTER 5: SYSTEM TESTING AND VALIDATION 31
5.1 System Testing 32
5.2 Testing Techniques 32
5.2.1 Unit Testing 32
5.2.2 Integration Testing 33
5.2.3 Black Box Testing 33
5.2.4 Performance Testing 33
5.3 Test Cases 33
5.3.1 Test Case 1: Stroke Capture 34
5.3.2 Test Case 2: Word Recognition 34
5.3.3 Test Case 3: Candidate Selection and Unicode Insertion 35
5.3.4 Test Case 4: Response Latency 36
5.3.5 Test Case 5: Offline Operation and Privacy 37

5.4 Non-Functional Requirements 38
5.4.1 Accuracy Results 38
5.4.2 Speed Results 38
5.4.3 Usability Results 38
5.4.4 Privacy Results 39
5.4.5 Size and Resource Use Results 39
5.4.6 Reliability and Portability Results 39

CHAPTER 6: CONCLUSION 40
6.1 Conclusion 41
6.2 Limitations and Future Work 41

APPENDIX – I 42

REFERENCES 44

LIST OF FIGURES

Figure	Caption	Page No.
4.1 :	Main Use Case Diagram	21
4.2 :	Activity Diagram	23
4.3 :	DFD Level 0	24
4.4 :	DFD Level 1	25
4.5 :	System Sequence Diagram	26
4.6 :	Sequence Diagram	27
4.7 :	Class Diagram	28
4.8 :	Layered Architecture Diagram	29
4.9 :	Interface Design Diagram	29
4.10 :	Component Level Design	30
4.11 :	Deployment Diagram	30

LIST OF TABLES

Table	Caption	Page No.
1.1 :	Hardware requirements	4
1.2 :	Software requirements	5
1.3 :	Dataset totals	5
2.1 :	Summary of Reviewed Literature	9
2.2 :	Summary of Existing Systems	10
4.1 :	Full Dress Use Case for Write Urdu Handwriting	21
4.2 :	Full Dress Use Case for Select Suggested Word	22
5.1 :	Test Case 1: Stroke Capture	34
5.2 :	Test Case 2: Word Recognition	34
5.3 :	Test Case 3: Candidate Selection and Unicode Insertion	35
5.4 :	Test Case 4: Response Latency	36
5.5 :	Test Case 5: Offline Operation and Privacy	37
5.6 :	Non-Functional Requirements, Measures and Results	38

LIST OF ACRONYMS

BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
CER	Character Error Rate
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
DFD	Data Flow Diagram
GAN	Generative Adversarial Network
GNN	Graph Neural Network
HWR	Handwriting Recognition
IME	Input Method Editor
NPU	Neural Processing Unit
ONNX	Open Neural Network Exchange
SDG	Sustainable Development Goal
SSD	System Sequence Diagram
SVM	Support Vector Machine
TFLite	TensorFlow Lite
WER	Word Error Rate

Chapter 1

INTRODUCTION

Rapid digitization has made smartphones a central piece in our life. It changes how we communicate, socialize and work. For Latin-script based languages, text entry on smartphones is not a problem. QWERTY keyboards and glide typing allow easy text input. Even natural handwriting support for many languages is now built-in in modern keyboards. There is no such support for Urdu which is spoken by more than 230 million people around the world and the national language of Pakistan along with being a recognized scheduled language in India.

Urdu is written in cursive and context sensitive Perso-Arabic script. One Urdu letter can take isolated, initial, medial or terminal shape depending on the context. These different forms of letters form 108 unique half-form letteres that must be recognized by the recognizer. Some academic progress has been made on this problem. A preprocessing pipeline which used preclassification and wavelet features by Safdar et et al. [1] successfully achieved 96.1% accuracy on half-form characters and some studies achieved between 85% to 93% accuracy on isolated characters[3, 6, 7, 8]. But these systems are limited to isolated characters. Commercial keyboards are somewhat better but support for Nastaliq style is limited. There is a gap for a mobile keyboard that can be used to recognized Urdu handwriting.

This report presents *Urdu-Ink Mobile*, a mobile and tablet keyboard that can recognize Urdu handwriting in real-time. Users write Urdu in a natural way, the keyboard recognizes the words and converts them into editable Unicode text usable in any application. First, It uses a Neural Network to recongize the words. Second, it introduces a GAN network [10]trained on the collected dataset to generate synthetic sampeles that can increase the total number of samples in the dataset. Third, this system runs completely on-device requiring no internet connection.

1.1 Project Domain

This project falls in the domain of artifficial intelligence, specifically deep learning and pattern recognition for online handwriting recognition. Online handwriting recognition differs from offline recognition as it used a realtime stream of stroke coordinates instead of scanned images. The language processing and recognitin part involves neural networks to recognize the words and the mobile deployment part includes packaging the trained model into a keyboard that runs withing the memory limits of smartphone devices. The problem that is solved in this project is the conversion of continuous Urdu handwriting into its corresponding Unicode text in real-time.

1.2 Problem Identification

Many contributions have been made in the field of Urdu character recognition. The most recent and some of the complete contribution has been made by by Safdar et al. [1], which achieved a 96.1% accuracy in recognizing 108 half-form characters using preclassification and wavelet feature. But these contributions are limited to isolated and short characters. Mobile input using fingers is also a problem due to small screen size and quick writing.

Commercial keyboards do not solve this prolem completely with their limited and basic suppport for Nastalic style Urdu handwriting. The main problem solved by our project is the absence of a word level, real-time and on-device Urdu handwriting recognizer keyboard.

1.2.1 Proposed Solution

Our proposed solution is *Urdu-Ink Mobile*, a system level mobile keyboard that records handwriting data in XY coordinates, time and pressure values and then passes that data through a pre-processing pipeline. The proprocessing pipeline applies jiter removal, down-sampling, interpolation/extrapolation and normalization to the raw handwriting data. A GAN network that is trained on the dataset is used to generate synthetic samples to increase the total number of samples. The preprocessed data is fed into a Neural Network that recognizes the handwritten words. The recognized words are then converted into Unicode text that can be used in any app.

1.2.2 Objectives

The objectives of the project are as follows.

1. To develop a multi-phase word level Urdu handwriting recognition pipeline for smartphones that builds upon the character recognition work of Safdar et al. [1]
2. To train a GAN network that generates synthetic handwriting samples to increase the total number of samples in the dataset
3. To collect a coordintes based Urdu handwriting dataset from native Urdu writers.
4. To protect user privacy and reduce latency by running all inference locally using TensorFlow lite runtime.
5. To test and evaluate the trained models on real mobile devices.

1.2.3 Scope of the Project

The main target of this system is to recognize handwritted Urdu words written in Nastaliq/Naskh style. The system is developed to run on Android 10 and later. The users write on the canvas inside the keyboard app, the trained model recognizes the word and converts it into editable Unicode text to be used in any app such as messaging or email. Recognizing other language words or decorative calligraphic styles in not included in the scope of our project. Educators, journalists, students and other general users are expected users of this system in routine tasks and communications.

1.3 Effectiveness / Usefulness of the System

Our system is different from existing systems as it brings word-level recognition and complete local inference. Our system can be used in any app because it is desinged as a keyboard that can give system-input. Native Urdu speakers who prefer to use Urdu instead of English

can use this system to conveniently write in their native language. It can be used by students for their assignments and note-taking and by geneal users for filling forms and performing other handwriting related tasks. Keeping the security and privacy in mind, the system does not send any handwriting data to the internet and all inference is performed on-device.

1.4 Resource Requirement

General consumer hardware is required for model training, GAN network training and on-device inference. Deep learning frameworks are required for model training. A dataset of coordinates based Urdu handwriting is required for training the recognizer. These requirements are described in more detail in the following subsections.

1.4.1 Hardware Requirement

Consumer grade hardware is required for model and GAN network training. The deployed keyboard can conveniently be run on Android 10+ smartphones. Table 1.1 lists the minimum specification of the target device, which was finalized in the project proposal. It also list the workstation required for training.

Table 1.1: Hardware requirements

Component	**Target device (minimum)**	**Training workstation**
Device	Android/iOS smartphone or tablet	HP/Mechrevo
Processor	Quad-core 1.8 GHz	Ryzen 5 7430U/ Intel I3 13100U
Memory (RAM)	3 GB	8 GB
Storage	500 MB	128
Accelerator	Mobile GPU/NPU recommended	Integrated GPU

1.4.2 Software Requirement

This system uses modern frameworks for training the Neural Network. Python 3.11 is used for data processing and model training, Java is used for Android keyboard development, and C++ for optimizing the model so that it runs smoothly on smartphones. Networks were trained using PyTorch 2.x, Keras for the GAN experiments, and TensorFlow Lite for optimized local inference.Table 1.2 provides a summary of all the software tools.

Table 1.2: Software requirements

Category	**Tools / Languages**
Programming languages	Python 3.11, Java, C++
Deep learning frameworks	PyTorch 2.x, TensorFlow Lite, Keras
Mobile inference runtimes	ONNX Runtime Mobile, CoreML, TensorFlow Lite
Keyboard user interface	Java
Target operating systems	Android 10+

1.4.3 Data Requirement

As the recognizer is a machine learning, it requires a dataset of relevant coordinates based Urdu handwriting samples. Each word is stored in a .bin file starting with a four-byte magic header, followed by the canvas width and height represented as 64-bit floating-point values and then the number of strokes as an unsigned 32-bit integer. Number of points are stored for every stroke. For each point, the values of x,y coordinates and pressure are stored as 64-bit floating points. An unsigned 32-bit integer is used to store the time offset. All these values use little endian order. Many writers contributes samples to the dataset. The dataset is split into training, validation and test sets, with separate writers for training and testing data samples. The dataset is augmented using a GAN network that was trained on the same samples.Total number of writers and samples in the dataset is listed in Table 1.3.

Table 1.3: Dataset totals

Measure	**Total**
Writers	12
Words	996
Samples	1,619

1.5 Report Organization

This report has been organized into six chapters. Chapter 1 introduces our project, its domain, problem, objectives, scope and resource requirements. Chapter 2 includes literature and review of existing systems. It identifies the research gap and defines the boundary of the proposed solution. Chapter 3 lists the modules, functional requirements and non-functional requirements of the system. Chapter 4 the system models and design are covered in this chapter, including the use case, activity, data flow, sequence, class and architectural diagrams. Chapter 5 goes over the testing techniques, test cases and the validation of the non-functional requirements. Chapter 6 includes the conclusion of the report and discusses limitations and future work. An appendix and the list of references is added at the end of the report.

Chapter 2

BACKGROUND AND EXISTING SYSTEMS

The problem under consideration is the real-time recognition of continuous Urdu handwriting written on a smartphone and its conversion into Unicode text. Urdu's cursive, context-dependent script makes this difficult, and existing academic and commercial solutions cover only parts of the problem.

This chapter is organized as follows. Section 2.1 reviews related academic literature, Section 2.2 describes related systems and applications, Section 2.3 discusses the problems identified from existing work, and Section 2.4 defines the boundary of the proposed solution.

2.1 Related Literature Review

This section discusses six papers that relate to the project either through the technique they use or through the problem they address.

Safdar et al. [1] propose the base methodology of this project. Their approach recognizes 108 Urdu half-forms written on a tablet by first classifying the input into a smaller group using simple stroke features, such as the number of strokes and the position of diacritics, and then extracting db2 wavelet features that are classified with a support vector machine. The pre-classification stage reduces confusion between visually similar shapes and keeps the final classifier small. The reported accuracy is 96.1%. The limitations are that the data come from tablets, no segmentation is performed to handle words, and the system works at the half-form level only. These limitations motivate the extensions made in this project.

Razzak et al. [3] combined fuzzy logic with hidden Markov models to recognize 1,800 Urdu ligatures written in the Nastaliq style and reported an accuracy of 87.6%. Working at ligature level is an important step towards connected writing because ligatures are the natural units of Urdu words. The method is, however, limited to ligatures from a fixed vocabulary and relies on handcrafted features, which restricts how well it can generalize to new writers or to words made of unseen ligature combinations.

Hussain and Khan [6] used a spatial–temporal neural network to recognize 15 Urdu ligatures and reported an accuracy of about 85%. Their work is notable because it exploits the temporal order of pen movement, which is a central property of online handwriting. The experiment was run on a very small set of ligatures, so the result gives limited evidence of how the method would behave on the full Urdu character set or on natural writing.

Malik and Khan [7] proposed a holistic approach for 39 isolated Urdu characters and achieved about 93% accuracy. The holistic strategy avoids segmentation by treating each character as a single pattern, which works for isolated characters but cannot be applied directly to connected cursive words, where the boundaries between characters are not clear. The limitation of this work is therefore that it is restricted to isolated characters.

Shahzad et al. [8] used nine geometric features to recognize 38 isolated Urdu characters with an accuracy of 92.8%. The study also reports that accuracy dropped sharply, to about 31%, when the system was tested on non-native writers. This finding is important for the present project because it shows how strongly handwriting recognizers depend on writer style, and it is the reason for writer-invariant training in the proposed system.

Naz et al. [2] used multi-dimensional long short-term memory (MDLSTM) networks to recognize offline Nastaliq text and obtained an accuracy of roughly 88%. The work shows that recurrent networks can model the long-range dependencies of cursive Urdu without explicit segmentation. It operates on scanned images of text, so there is no interactive input, no use of pen-trajectory information and no real-time operation, which makes it unsuitable for a keyboard.

Beyond the Urdu-specific studies, the proposed pipeline rests on general deep learning techniques whose original papers were also reviewed. Goodfellow et al. [9] introduced generative adversarial networks, in which a generator that produces samples and a discriminator that tries to tell them apart from real data are trained against each other, so that the generator gradually produces output that is difficult to distinguish from genuine data. This idea underlies the RefinerGAN of the present project. Zhu et al. [10] proposed CycleGAN, which learns a mapping between two collections of data without paired examples by enforcing cycle consistency, and which is useful here because paired samples of the same word written by different writers do not exist. Vaswani et al. [11] replaced recurrence with self-attention in the Transformer, which allows every part of a sequence to be related to every other part and makes training easy to parallelize. Graves et al. [12] introduced connectionist temporal classification, a loss that lets a recurrent network be trained on unsegmented sequences without knowing which input frames belong to which output character, and the thesis of Graves [5] describes it in the wider context of sequence labelling. Devlin et al. [13] showed that a deep bidirectional Transformer pretrained on large text collections can be adapted to many language tasks, which is the basis of the BERT-Urdu rescoring. Finally, Ganin et al. [14] proposed domain-adversarial training, in which a network learns features that are useful for the main task but do not reveal the domain of the input; in this project the domain is the writer, so the method supports writer-independent recognition. Abandah and Jamour [4] showed that skeleton-based grapheme segmentation is an efficient way of splitting handwritten Arabic-script words, a technique that carries over to Urdu ligature segmentation. None of these works is specific to Urdu on mobile devices, and the present project combines them for that purpose.

Table 2.1 summarizes the Urdu-specific papers that were reviewed.

Table 2.1: Summary of Reviewed Literature

Year	Authors	Contribution	Techniques	Limitations / Remarks
2022	Safdar et al.	108 half-forms, 96.1% accuracy	Pre-classification, db2 wavelet, SVM	Tablet only; no segmentation; no word level
2010	Razzak et al.	1,800 ligatures, 87.6% (Nastaliq)	Fuzzy logic, HMM	Limited to fixed ligatures
–	Hussain & Khan	15 ligatures, 85%	Spatial–temporal neural network	Tiny dataset
–	Malik & Khan	39 isolated characters, 93%	Holistic recognition	Isolated characters only
–	Shahzad et al.	38 isolated characters, 92.8%	Nine geometric features	Falls to 31% for non-native writers
2014	Naz et al.	Offline Nastaliq text, about 88%	MDLSTM	Offline; no interactivity

2.2 Related Systems/Applications

Several commercial and open-source keyboards offer handwriting or phonetic input, and they were reviewed to understand what is available to Urdu users today.

Gboard, the keyboard developed by Google, supports handwriting input for many languages and works system-wide on mobile devices. Its Urdu support is basic and concentrates on Naskh-style writing, and it has difficulty with connected Nastaliq-style cursive. The advantage of Gboard is its wide availability, and its limitation is the quality of Urdu handwriting recognition. [Verify the current Gboard Urdu handwriting behavior on your test device before submission.]

SwiftKey, developed by Microsoft, is best known for predictive text and personalized word suggestions and is available as a system-wide keyboard. Its handwriting support for Urdu is limited, so it helps users who type but not those who prefer to write. Its strength is its language model, which is a design idea that is also used in the proposed system.

The default Apple iOS keyboard provides handwriting recognition for English and Chinese, but it provides no Urdu handwriting recognition. Urdu users on iOS must therefore rely on typing with a fixed layout or on third-party applications.

Easy Urdu is a phonetic typing application, in which the user writes Urdu words by typing Roman letters that are converted into Urdu script. It is helpful for users who are comfortable with Roman characters, but it does not offer handwriting recognition.

Table 2.2 compares these systems.

Table 2.2: Summary of Existing Systems

Year	System	Contribution	Tools / Technologies	Limitations	Applications
Current	Gboard (Google)	Multi-language handwriting recognition	Neural recognizers, system keyboard	Basic Naskh support; weak on Nastaliq	Messaging, general text
Current	SwiftKey (Microsoft)	Predictive text	Language models, system keyboard	Limited handwriting support	Messaging, general text
Current	Apple iOS Keyboard	English and Chinese handwriting	System keyboard	No Urdu handwriting	Messaging, general text
Current	Easy Urdu	Phonetic typing	Roman-to-Urdu mapping	No handwriting recognition	Messaging

2.3 Identified Problem from Existing Work

The literature and the existing keyboards show a clear pattern of limitations. From a technology point of view, academic systems recognize isolated characters, half-forms or ligatures, and use handcrafted features and classical classifiers such as support vector machines and hidden Markov models. Their datasets are small, their accuracy drops for writers not seen in training, as shown by the fall to 31% reported for non-native writers [8], and they were tested on tablets and not on the small, noisy touchscreens of phones. The only offline approach that scales to text, based on MDLSTM [2], has no interactivity and works on scanned images. From a functional point of view, none of the reviewed systems offers word-level recognition of continuous cursive, system-wide use in any application, or language-model correction of recognition errors. From a non-functional point of view, commercial keyboards give only basic Urdu support, and none of the academic systems report real-time latency or privacy protection on a mobile device. These gaps can be addressed by extending the pipeline of Safdar et al. [1] with stroke refinement through a GAN, ligature segmentation, modern sequence models, language-model post-processing and on-device inference, and these gaps define the scope of this project.

2.4 Selected Boundary for Proposed Solution

The proposed solution implements the following functionality, each part of which responds to a gap identified in Section 2.3. A system-level Input Method Editor captures strokes on the touchscreen of a phone or tablet so that recognition is available in every application. A preprocessing engine and a RefinerGAN network clean noisy strokes, which responds to the lack of robustness on mobile input. A skeleton-based segmenter and a graph-neural-network pre-classifier handle continuous cursive, so that recognition moves from half-forms to ligatures and words. A hybrid feature extractor and a Transformer with BiLSTM-CTC decoder

replace the classical classifiers, while an N-gram model and BERT-Urdu correct errors using context. A CycleGAN enlarges the corpus and domain-adversarial training makes the recognizer less dependent on the writer. All models run on the device through TensorFlow Lite or ONNX Runtime so that the keyboard works offline and keeps handwriting private.

Several things are not implemented in the project. The system does not recognize scripts other than Urdu or highly decorative calligraphic styles, because they would require additional training data. Real-time performance depends on the mobile GPU or NPU, so the system cannot guarantee smooth operation on very old entry-level phones. Optional accessibility add-ons such as translation require an internet connection and are outside the offline core. Full deployment on every Linux-based mobile operating system and the recognition of unconstrained long documents are left for future work.

Chapter 3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

This chapter describes what Urdu-Ink Mobile must do and how well it must do it. It covers the specification of the system, the modules into which the system is divided, the functional requirements that define its behavior and the non-functional requirements that define its quality. All requirements are aligned with the scope and objectives given in Chapter 1.

The chapter is organized as follows. Section 3.1 gives the system specification, Section 3.2 describes the system modules, Section 3.3 lists the functional requirements, and Section 3.4 describes the non-functional requirements together with the way each one is measured.

3.1 System Specification

The system specification fixes the technical boundaries within which the keyboard is designed. Urdu-Ink Mobile recognizes cursive, context-sensitive Nastaliq/Naskh Urdu writing that includes more than 108 half-form shapes. It operates on smartphones and tablets running Android 13 or later and iOS 16 or later, with a possible port to Linux-based mobile operating systems. It works as a real-time, system-level keyboard, meaning that it is installed as an Input Method Editor and is available in every application. The recognition granularity covers half-forms, ligatures, words and sentences. The specification of the target device is a quad-core 1.8 GHz processor, 3 GB of RAM and 500 MB of storage, as listed in Table 1.1. This chapter explains the modules, functions and quality attributes that follow from this specification.

3.2 System Modules

A module is a separate unit of software that accomplishes one function and contains everything necessary to do so. Splitting the keyboard into modules allows each stage of the recognition pipeline to be developed, trained, replaced and tested independently, and it allows the heavy models to be optimized separately for mobile deployment. The system is divided into eleven modules, which follow the order in which handwriting flows through the pipeline from the touchscreen to the Unicode output.

3.2.1 Data Acquisition Module

This module captures the raw handwriting. A specialized Input Method Editor interface reads touch events directly from the capacitive touchscreen, and a stroke logger records the coordinates, pressure and timing of every point of every stroke, including strokes that are written close together in time. The same module is used during data collection to record the training corpus in the binary stroke format described in Section 1.4.3, and during normal use to feed the live recognition pipeline.

3.2.2 Urdu Handwriting Corpus Module

This module stores and manages the handwriting data used to train and evaluate the models. Samples are gathered from different native writers and cover half-forms, ligatures and words in continuous cursive. To increase the amount and diversity of the data, a CycleGAN generates synthetic handwriting samples in the style of real writers. The module keeps training, validation and test partitions apart, and keeps the writers of the test set separate from those of the training set so that writer-independent accuracy can be measured honestly.

3.2.3 Preprocessing Engine

The preprocessing engine improves the raw ink before it reaches the neural networks. Resampling places points at a uniform distance from each other, and smoothing removes the jagged edges that come from finger movement. Normalization adjusts scale and pressure so that the data do not depend on the size of the writing or the force used by the writer. Denoising removes accidental touches and electronic interference. This standardization lets all later modules work on consistent input.

3.2.4 RefinerGAN Module

The RefinerGAN module addresses the messy nature of handwriting on a small screen. It is a generative adversarial network whose generator learns to map irregular, jittery user strokes to a clean latent space of standardized Nastaliq ligatures. During training, a discriminator checks that the refined strokes still resemble genuine Urdu script, so that the refinement improves recognizability without changing the linguistic meaning of what was written. At run time only the generator is deployed on the phone.

3.2.5 Ligature Segmentation Module

Urdu words are written as connected ligatures, so recognition requires a way to divide a word into manageable parts. This module uses a skeleton-based approach: it thins the written word to its skeleton and then detects the natural break points of the cursive script, so that the word is split into ligatures or characters, in the manner of skeleton-based grapheme segmentation for cursive Arabic-script writing [4]. Grapheme detection identifies where one unit ends and the next begins, which improves the accuracy of the downstream recognition.

3.2.6 Pre-Classifier Module

The pre-classifier reduces the work of the main recognizer. A graph neural network encodes the topology of the strokes, that is, their structure and connectivity, together with simple features such as the number of strokes and the position of dots and diacritical marks. On the basis of this encoding, the input is routed to one of 28 subsets of shapes, so that the recognition core only has to choose among the members of one subset instead of among all 108 half-forms. This lowers the computational load and reduces confusion between similar-looking shapes.

3.2.7 Hybrid Feature Extraction Module

This module turns each segmented ligature into a numerical representation that combines three kinds of information. The db2 wavelet transform captures the mathematical shape of the trajectory at several scales, structural features describe properties such as loops, dots and direction changes, and learned CNN embeddings capture visual patterns that are hard to describe by hand. The three representations are combined into a single vector so that the classifier benefits from both handcrafted and learned knowledge.

3.2.8 Recognition Core

The recognition core is the classifier of the system. A Transformer encoder models the temporal order and long-range dependencies of the handwriting, and a BiLSTM with a CTC decoder converts the variable-length sequence into a sequence of characters without requiring an exact alignment between input and output [12, 5]. Domain-adversarial training [14] is applied so that the learned features depend as little as possible on the identity of the writer. This arrangement replaces the support vector machines and simple recurrent networks of earlier work and is well suited to the sequential nature of cursive Urdu.

3.2.9 Language Model Layer

Recognition errors that come from visual ambiguity are corrected using linguistic context. An Urdu N-gram model scores the probability of word sequences, and BERT-Urdu rescoring, based on the BERT architecture [13], applies deeper contextual understanding to choose the most likely word among visually similar candidates. The layer also produces the ranked list of candidate words shown to the user.

3.2.10 Real-Time Inference Engine

The inference engine runs the trained models on the phone. The models are converted to TensorFlow Lite, ONNX or CoreML formats, quantized to reduce size, and executed through the corresponding mobile runtime with the help of the mobile GPU or NPU where it is available. Running everything locally keeps latency low, makes the keyboard work without a network connection and keeps the handwriting private.

3.2.11 Evaluation and Analytics Module

This module measures how well the system works. It calculates Character Error Rate (CER) and Word Error Rate (WER) against labelled test samples and records latency measurements. A performance dashboard shows a confusion matrix that lets developers see which characters are most often misidentified, so that the corpus or the models can be improved.

3.3 Functional Requirements/Software Features

Functional requirements state what the system must do. The requirements below are derived from the objectives and scope of the project and are each linked to one or more of the

modules in Section 3.2. They are used in Chapter 5 to design the test cases.

3.3.1 Stroke Capture (FR1)

The system shall capture finger or stylus strokes on a writing canvas inside the keyboard, recording the position, pressure and time of every point, and shall support multiple strokes for a single word, including late-written dots and diacritical marks.

3.3.2 Stroke Preprocessing (FR2)

The system shall resample, smooth, normalize and denoise captured strokes so that the recognition stages receive input of uniform quality regardless of the speed, size or pressure of the writing.

3.3.3 Stroke Refinement (FR3)

The system shall pass preprocessed strokes through the RefinerGAN generator, which returns cleaner strokes that keep the linguistic identity of the written shapes.

3.3.4 Segmentation and Pre-classification (FR4)

The system shall divide continuous cursive writing into ligatures using skeleton-based segmentation and shall route each ligature to one of the 28 subsets by using the graph neural network.

3.3.5 Handwriting Recognition (FR5)

The system shall recognize half-forms, ligatures and words of Urdu cursive handwriting by using the hybrid feature extractor and the Transformer with BiLSTM-CTC classifier, and shall output a character sequence for each written word.

3.3.6 Language-Model Correction and Suggestions (FR6)

The system shall correct recognition errors using the N-gram model and BERT-Urdu, and shall display a ranked list of candidate words from which the user can choose.

3.3.7 Unicode Insertion (FR7)

The system shall insert the accepted word into the text field of the active application as Unicode text, so that the result is editable and searchable by that application.

3.3.8 Text Correction and Editing (FR8)

The system shall allow the user to delete the last character or word, to clear the canvas, to rewrite a word and to insert a space or a new line without leaving the keyboard.

3.3.9 Keyboard Management (FR9)

The system shall be installable as a system keyboard, shall let the user switch between the handwriting keyboard and other keyboards, and shall provide a settings screen for choices such as recognition suggestions and writing speed.

3.3.10 Performance Evaluation (FR10)

The system shall provide an evaluation mode in which CER, WER and latency are calculated for a labelled test set and in which a confusion matrix is displayed for developers.

3.4 Non-Functional Requirements

Non-functional requirements, also called quality attributes, describe how well the system performs its functions. Only the attributes that matter for a real-time handwriting keyboard are included, and each of them is stated with a rationale and a measure so that it can be verified in Chapter 5. The numeric thresholds below are the targets set for the project.

3.4.1 Recognition Accuracy

A keyboard that makes frequent mistakes forces the user to correct text and is quickly abandoned, so recognition accuracy is the most important quality attribute. It is measured as Character Error Rate and Word Error Rate on a test set written by writers who were not part of the training data. The target is a CER of at most 8% and a WER of at most 20% for unseen writers, and a half-form accuracy that is not lower than the 96.1% reported in the base work [1] on comparable data.

3.4.2 Speed

Handwriting recognition feels natural only when the result appears almost immediately after the user stops writing. Speed is measured as the time between the end of a word (the pen lifting and a short pause) and the display of the candidate list on the reference device with the minimum specification. The target is an average of at most 300 ms and a 95th-percentile value of at most 600 ms.

3.4.3 Usability

Usability is the ease with which a person learns and uses the keyboard. It is important because the target users include people who are not comfortable with English keyboards. It is measured by observing new users and by a System Usability Scale questionnaire. The keyboard is considered usable if a new user can write and insert a first word within two attempts after a one-minute demonstration, and if the average System Usability Scale score is at least 70.

3.4.4 Privacy

Handwriting is personal data and can reveal what a person writes in private messages. The system therefore performs recognition entirely on the device. Privacy is measured by monitoring network traffic while the keyboard is used in offline-core mode, and the requirement is met if no handwriting, stroke or recognized text is sent over the network.

3.4.5 Size and Resource Use

A keyboard that uses too much storage or memory cannot run on low-cost phones. Size is measured in megabytes of installed storage and the peak memory used during recognition. The installed application with all models shall not exceed 500 MB of storage, which is the minimum storage figure of the target device, and the peak memory use shall stay within the 3 GB available on the reference device.

3.4.6 Reliability and Robustness

Because the keyboard is used in every application, a crash would interrupt the work of the user in other applications. Reliability is measured as the number of failures during a long run of continuous use, and robustness as the time required to recover from a failure. The target is no crash during 1,000 consecutive recognized words and a restart after failure in less than two seconds without loss of the text already inserted.

3.4.7 Portability

The system is intended to serve users of both major mobile platforms. Portability is measured by the number of target operating systems on which the keyboard is installed and passes the functional tests, and by the proportion of recognition code shared between platforms. The requirement is to pass the functional tests on Android 13 or later and iOS 16 or later, with the same trained models used on both.

Chapter 4

SYSTEM MODELING AND DESIGN

System design and analysis translate the requirements of Chapter 3 into models that developers and reviewers can examine before and during implementation. Urdu-Ink Mobile is an interactive system with a user who writes on the screen, a pipeline of processing stages, and a host application that receives the result, so the models in this chapter show the functions available to the user, the flow of activities, the flow of data, the interaction between the user and the system, the interaction between the internal objects, the software classes, and the architecture of the whole system.

The chapter is organized as follows. Section 4.1 gives an overview of the models used, Section 4.2 presents the use case diagram, Section 4.3 gives the full dress use cases, Section 4.4 presents the activity diagram, Section 4.5 the data flow diagrams, Section 4.6 the system sequence diagram, Section 4.7 the sequence diagram, Section 4.8 the design class diagram, and Section 4.9 the architectural diagrams.

4.1 System Design and Analysis

Analysis models describe what the system must do, and design models describe how it is built. In this project the following models are used. Use case diagrams and full dress use cases describe the functions seen from the point of view of the user. An activity diagram models the workflow of a recognition session. Data flow diagrams of level 0 and level 1 show how handwriting data are transformed into Unicode text. A system sequence diagram and a sequence diagram show the interaction between the user and the system and between the internal objects. A design class diagram shows the main software classes, and architectural diagrams, which include a layered architecture, the interface design, the component-level design and the deployment diagram, show the structure of the final product. Not every possible diagram is drawn, only those that add information for a system of this nature.

4.2 Use Case Diagrams

A use case diagram depicts the functions of the system that are performed through interaction with its users, and it helps to view the system from the perspective of the user. Figure 4.1 shows the main use case diagram of Urdu-Ink Mobile. The primary actor is the User, who writes Urdu words on the canvas, reviews and selects suggested words, corrects text and configures the keyboard. Writing a word includes the recognition of the strokes and the insertion of the accepted text into the host application. A second actor, the Developer/Researcher, collects handwriting samples and views the evaluation dashboard.

4.3 Full Dress Use Case/Detailed Use Case

Full dress use cases describe the main scenario and the alternatives of a use case in detail, and they are written only for use cases that are complex and need explanation. Two use cases were chosen for detailing: Write Urdu Handwriting, because it contains the whole recognition pipeline and most of the project's functional requirements, and Select Suggested Word, because it shows how language-model suggestions are used to resolve ambiguity.

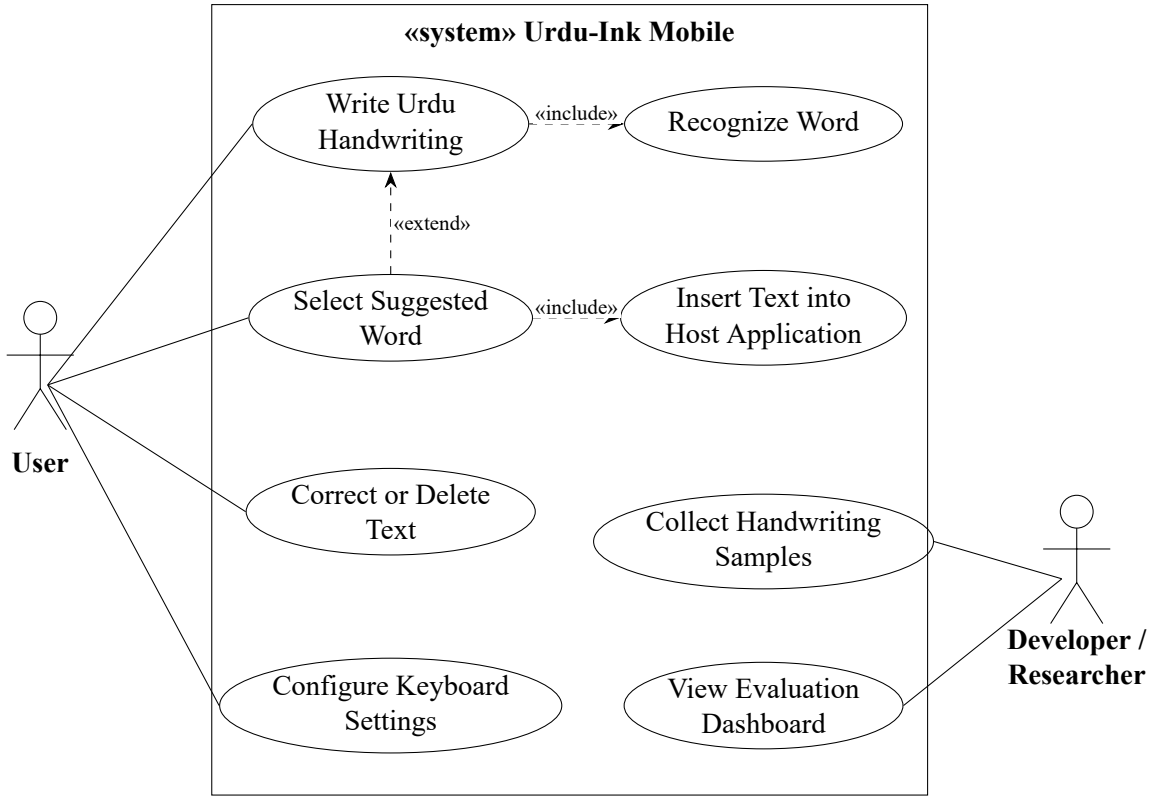


Figure 4.1: Main Use Case Diagram

Simple use cases such as configuring the settings are not detailed.

4.3.1 Write Urdu Handwriting – Full Dress Use Case

Table 4.1 describes the use case in which a user writes a word on the canvas and the system produces candidate words.

Table 4.1: Full Dress Use Case for Write Urdu Handwriting

Use Case Name	Write Urdu Handwriting
Scope	Urdu-Ink Mobile keyboard
Level	User goal
Primary Actor	User
Stakeholders and Interests	User: wants fast and correct conversion of handwriting into text. Application owner: wants the host application to receive valid Unicode. Developer: wants accurate logs for evaluation.
Pre-conditions	The keyboard is installed and selected as the active input method; a text field of a host application has focus; the models are loaded.
Success Guarantee	A ranked list of candidate words for the written word is shown, and the handwriting data stay on the device.

Main Success Scenario	1. The user writes a word in continuous cursive on the canvas. 2. The system captures the strokes. 3. The system resamples, smooths and normalizes them. 4. The RefinerGAN cleans the strokes. 5. The system segments the word into ligatures and routes them with the pre-classifier. 6. The recognition core produces character sequences. 7. The language model rescores the candidates. 8. The system displays the ranked candidate words.
Extensions	2a. Accidental touch: the system discards the stroke as noise. 6a. Recognition confidence is low: the system shows the best candidates with a prompt to rewrite. 7a. Word not in the vocabulary: the system shows the raw recognition result.
Special Requirements	Candidates appear within 300 ms (Section 3.4.2); all processing is local (Section 3.4.4).

4.3.2 Select Suggested Word – Full Dress Use Case

Table 4.2 describes the selection of a word from the candidate list and its insertion into the host application.

Table 4.2: Full Dress Use Case for Select Suggested Word

Use Case Name	Select Suggested Word
Scope	Urdu-Ink Mobile keyboard
Level	User goal
Primary Actor	User
Stakeholders and Interests	User: wants the intended word inserted with a single tap. Host application: wants correct Unicode text.
Pre-conditions	A candidate list is displayed for the last written word.
Success Guarantee	The selected word is inserted into the active text field as Unicode and the canvas is cleared for the next word.
Main Success Scenario	1. The user taps the intended word in the candidate bar. 2. The system sends the Unicode text to the host application. 3. The system clears the canvas and the candidate bar. 4. The system updates the language-model context with the chosen word.
Extensions	1a. The intended word is not in the list: the user rewrites the word or deletes it. 2a. The host application rejects the input: the system shows an error and keeps the word in the candidate bar.
Special Requirements	Insertion completes within 100 ms of the tap (Section 3.4.2).

4.4 Activity Diagram

An activity diagram is a flowchart that shows the flow from one activity to another, and it is used here to give a high-level understanding of the system's functions from the perspective

of the recognition session. Figure 4.2 shows the flow of a session. The user writes strokes, which are preprocessed, refined, segmented, pre-classified and recognized, after which the language model rescores the result and candidate words are displayed. If the user accepts a candidate, its Unicode text is inserted into the host application, and otherwise the user edits or rewrites the word and the flow returns to the beginning. The decision node after the candidate list is the only point at which the flow branches, and the loop it creates matches the real behavior of a keyboard, where a word that is not recognized correctly is simply written again. All activities before the decision are performed automatically by the system without user action, and they are the activities that must complete within the latency target of Section 3.4.2.

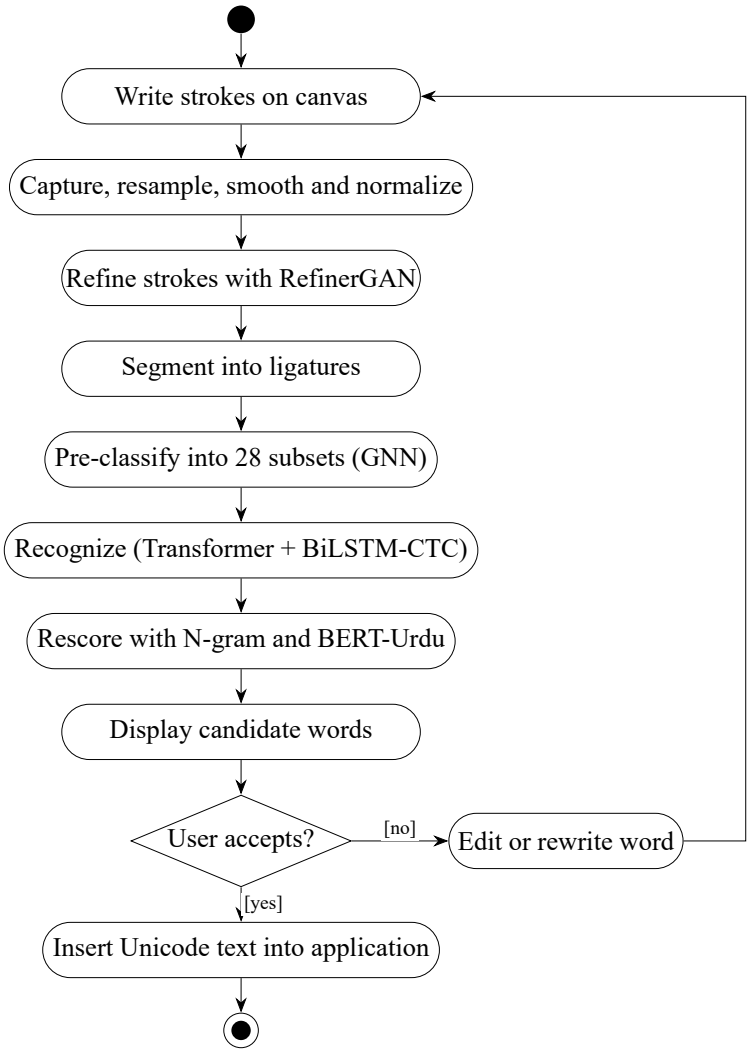


Figure 4.2: Activity Diagram

4.5 Data Flow Diagram

A data flow diagram represents the flow of data through a system, showing the inputs and outputs of each entity and process without control flow or decisions. A data flow diagram is appropriate for this project because the system receives input data in the form of strokes,

transforms them through a number of processes and returns text as output. Figure 4.3 shows the level 0 diagram, in which the whole keyboard is a single process that exchanges data with three external entities. The User supplies handwritten strokes and receives candidate words, the Host Application receives Unicode text, and the Developer/Researcher supplies test samples and receives evaluation metrics.

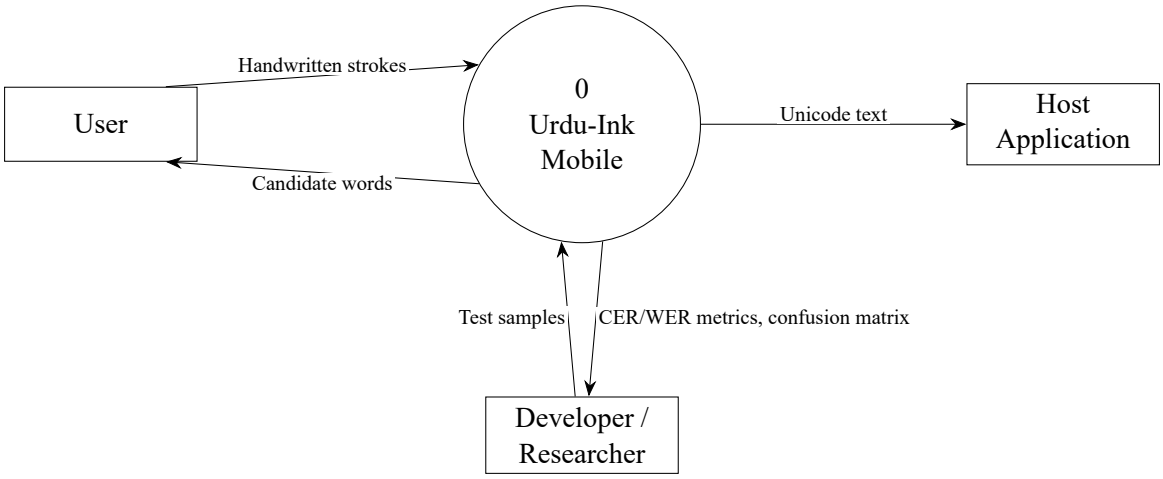


Figure 4.3: DFD Level 0

Figure 4.4 decomposes the system into six processes. Strokes from the user are captured and preprocessed (1.0), refined by the RefinerGAN (2.0), segmented and pre-classified (3.0), and recognized by the feature extractor and classifier (4.0). The language model (5.0) rescores the result and sends Unicode text to the host application, and an evaluation process (6.0) compares the recognized text against ground-truth labels. The models used by processes 2.0, 3.0 and 4.0 are held in the trained model store (D1), the language model process reads the Urdu text corpus (D2), and the evaluation process reads the labelled test samples (D3).

4.6 System Sequence Diagram

A system sequence diagram illustrates the input and output events that pass between an actor and the system treated as a black box, and it is derived from a use case. It is drawn here for the Write Urdu Handwriting and Select Suggested Word use cases of Section 4.3, so that the external events that the keyboard must handle are clear. Figure 4.5 shows that the user repeatedly sends stroke data while writing, signals the end of a word by lifting the finger and pausing, receives the list of candidates, selects one of them and finally causes Unicode text to be committed.

4.7 Sequence Diagram

A sequence diagram depicts the interaction between objects in the order in which messages are exchanged. It is drawn here for the event of recognizing and inserting one written word. Figure 4.6 shows that the input method service passes the captured strokes to the preproces-

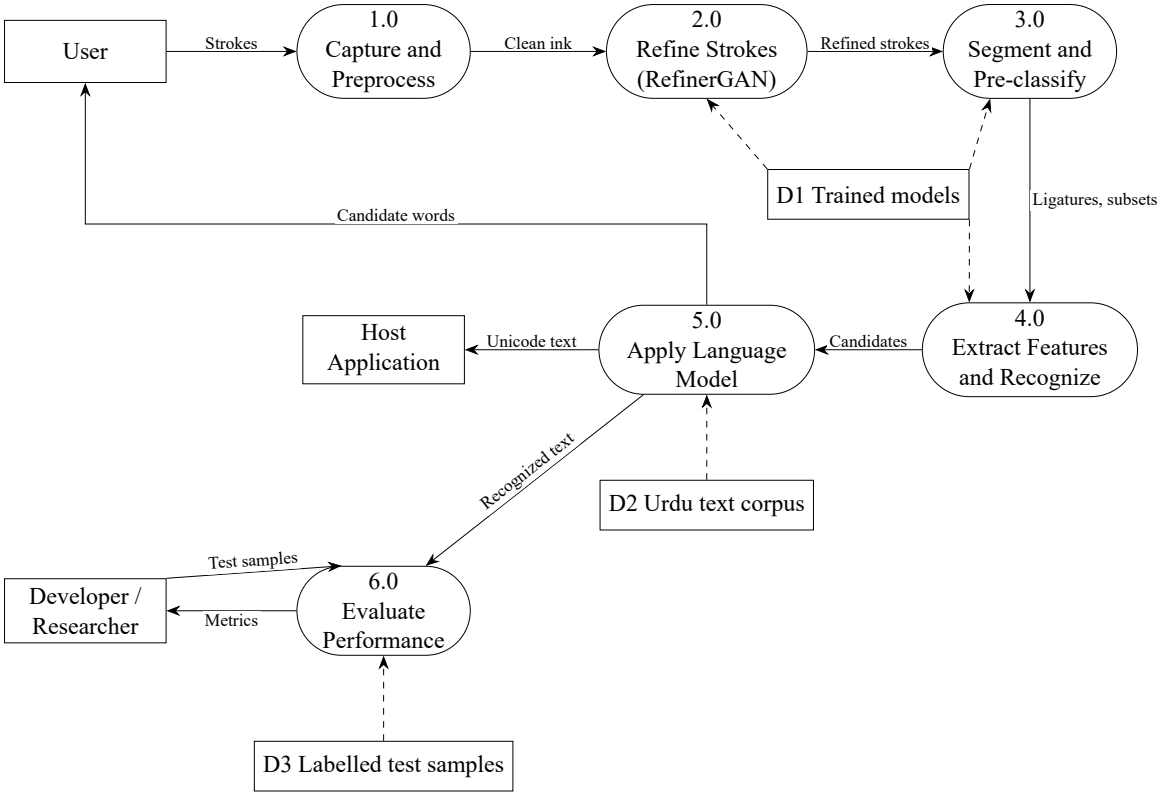


Figure 4.4: DFD Level 1

sor, which sends clean strokes to the RefinerGAN. The refined strokes are segmented, the ligatures are recognized, and the N-best list is rescored by the language model, after which the candidates are returned to the input method service and displayed to the user. When the user selects a word, the service commits the Unicode text to the host application.

4.8 Design Class Diagram

The design class diagram shows the main software classes of the system, their attributes and operations, and the relationships between them. The central class is UrduInkIME, the input method service that receives touch events, owns the InkSession (which holds the Stroke objects of the current word) and delegates recognition to the InferenceEngine. The InferenceEngine coordinates the pipeline classes in order: Preprocessor, RefinerGAN, LigatureSegmenter, PreClassifier, RecognitionCore and LanguageModel. Each of these classes corresponds to a module of Section 3.2. The UrduInkIME class is the only class that interacts with the operating system, which keeps the platform-specific Kotlin and Swift code separate from the recognition code. The InkSession aggregates one or more Stroke objects, and each Stroke in turn holds the ordered points with their coordinates, pressure and time offsets, mirroring the binary corpus format of Section 1.4.3. The InferenceEngine hides the differences between the mobile runtimes, so that a model can be exchanged without changes to the other classes. The numbers on the arrows between the pipeline classes show the order in which they are called for each written word. Figure 4.7 shows the diagram.

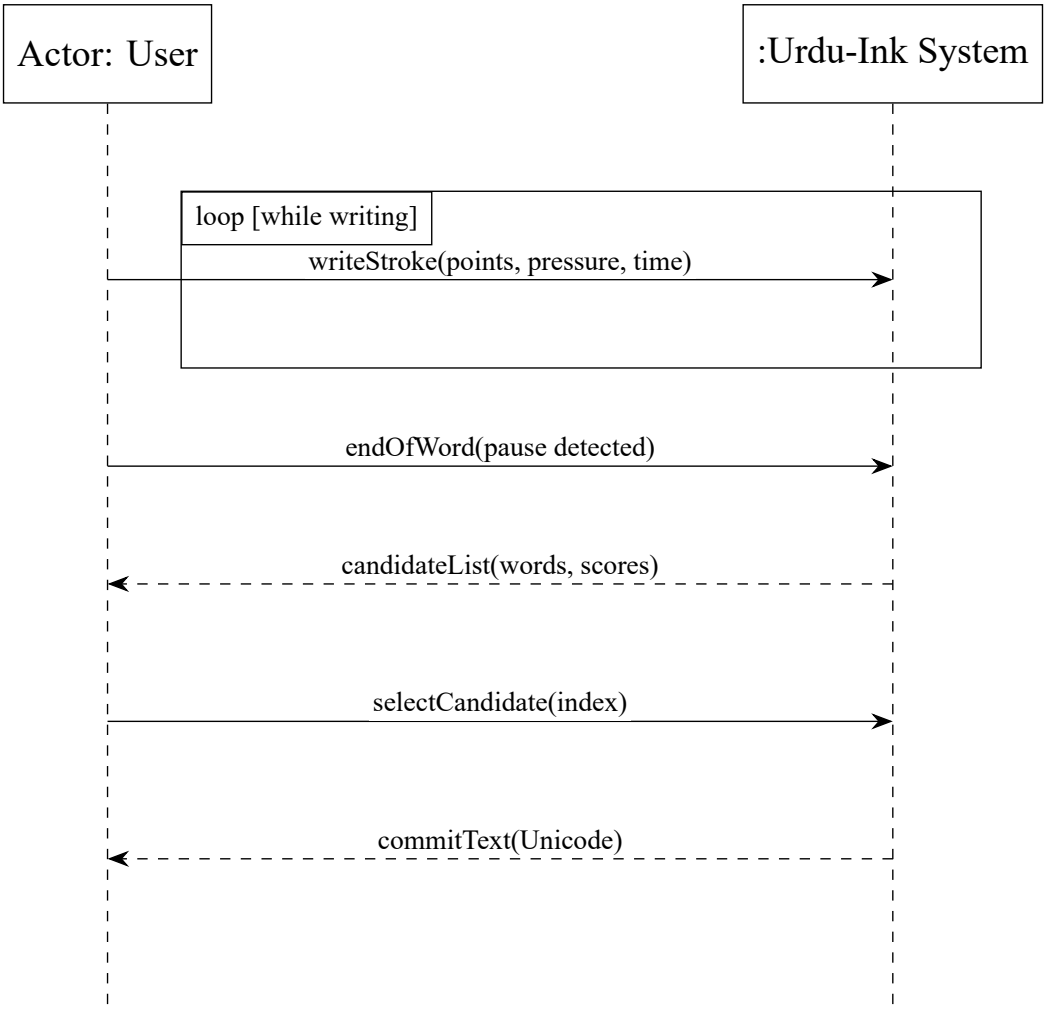


Figure 4.5: System Sequence Diagram

4.9 Architectural Diagrams

An architecture serves as a blueprint of the system. It provides an abstraction that manages complexity and defines how the components communicate and coordinate. A layered architecture is followed for Urdu-Ink Mobile because the processing stages form a natural pipeline and because each layer can be replaced or optimized without changing the others, for example when a new runtime or a better recognizer becomes available. Figure 4.8 shows the five layers of the system. The presentation layer holds the keyboard user interface. The input layer captures the strokes. The processing layer cleans, refines, segments and pre-classifies them. The recognition and language layer extracts features, recognizes and corrects the text. The runtime and platform layer executes the models on the operating system of the phone. The strict ordering of the layers also supports testing, because each layer can be exercised with recorded input from the layer above it, and it supports portability, because only the presentation, input and platform layers need platform-specific code while the processing, recognition and language layers use the same model files on Android and iOS. The interface design, component-level design and deployment diagram follow in the next subsections.

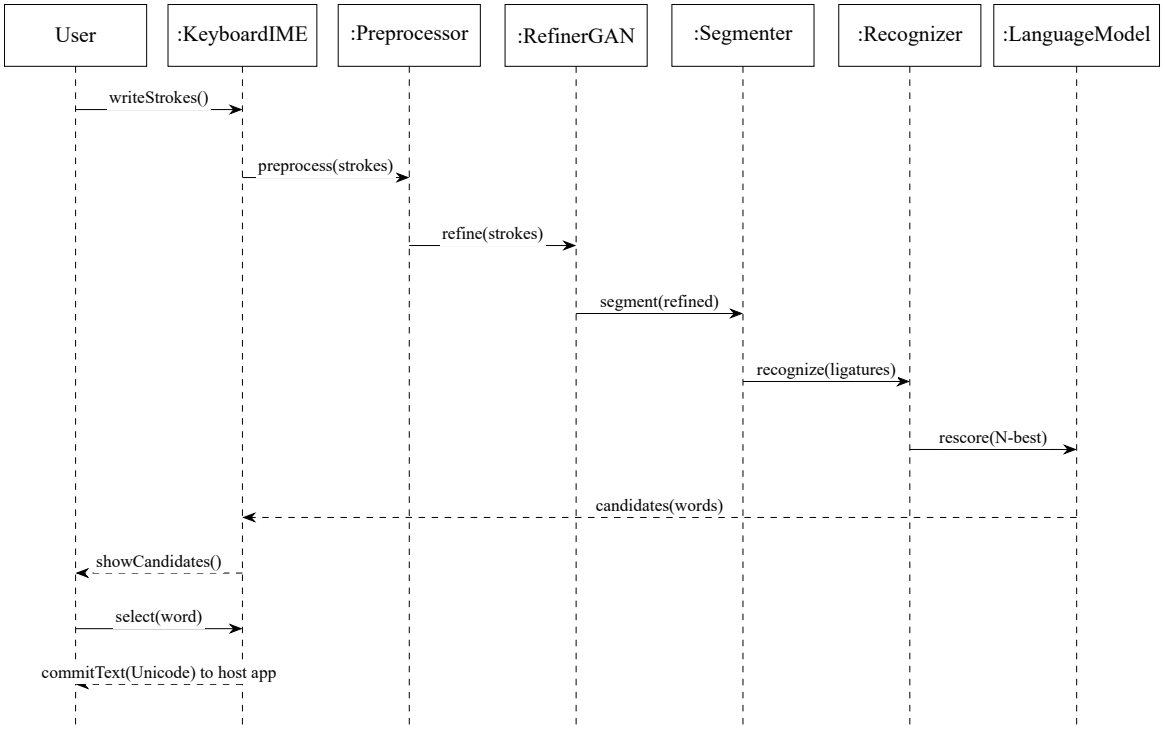


Figure 4.6: Sequence Diagram

4.9.1 Interface Design

The interface design shows the layout of the keyboard as the user sees it. Figure 4.9 presents the wireframe. The upper part of the screen shows the text field of the host application, below it a candidate bar displays the ranked words, and the largest area is the writing canvas with a dashed baseline that helps users write on a line. A bottom row holds the keys for switching keyboards, inserting a space, deleting and inserting a new line, and a button for the settings. The design keeps all common actions within reach of the thumb and allows the user to write and select a word without leaving the keyboard.

4.9.2 Component Level Design

The component diagram shows the software components and the interfaces through which they communicate. In Figure 4.10 the keyboard user interface passes strokes through the stroke-input interface to the Ink Capture component, which delivers raw ink to the Preprocessing Engine. The cleaned ink goes to the RefinerGAN component, then to the Ligature Segmenter, the Recognition Core and the Language Model component, and the final candidates return to the keyboard user interface. The three neural components, the RefinerGAN, the Recognition Core and the Language Model, obtain their model execution from the Inference Runtime component, which hides the differences between TensorFlow Lite, ONNX Runtime Mobile and CoreML.

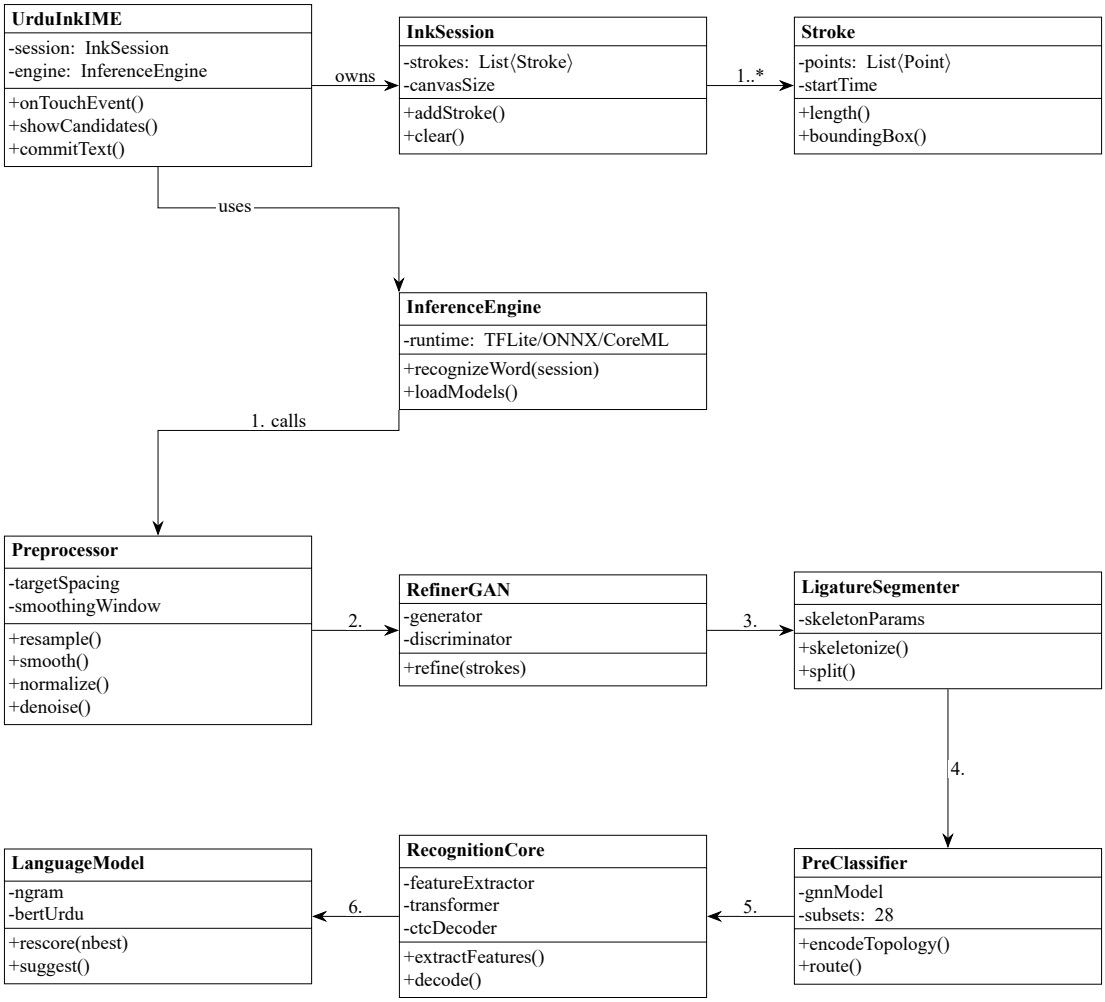


Figure 4.7: Class Diagram

4.9.3 Deployment

The deployment diagram shows the physical nodes on which the software runs. Figure 4.11 shows two nodes. Models are trained on a GPU workstation, where the Python training scripts and the handwriting corpus reside, and are exported to mobile formats. The exported models are bundled into the application installed on the smartphone, where the keyboard application, the model files, the inference runtime and a local store of user settings reside together. The keyboard communicates with host applications on the same phone through the input method framework of the operating system, and no handwriting data have to leave the phone. Keeping training and deployment on separate nodes also reflects the different needs of the two phases: training requires a powerful GPU and the complete corpus, while deployment requires only the compact, quantized model files and a runtime that fits within the memory and storage limits of the reference device.

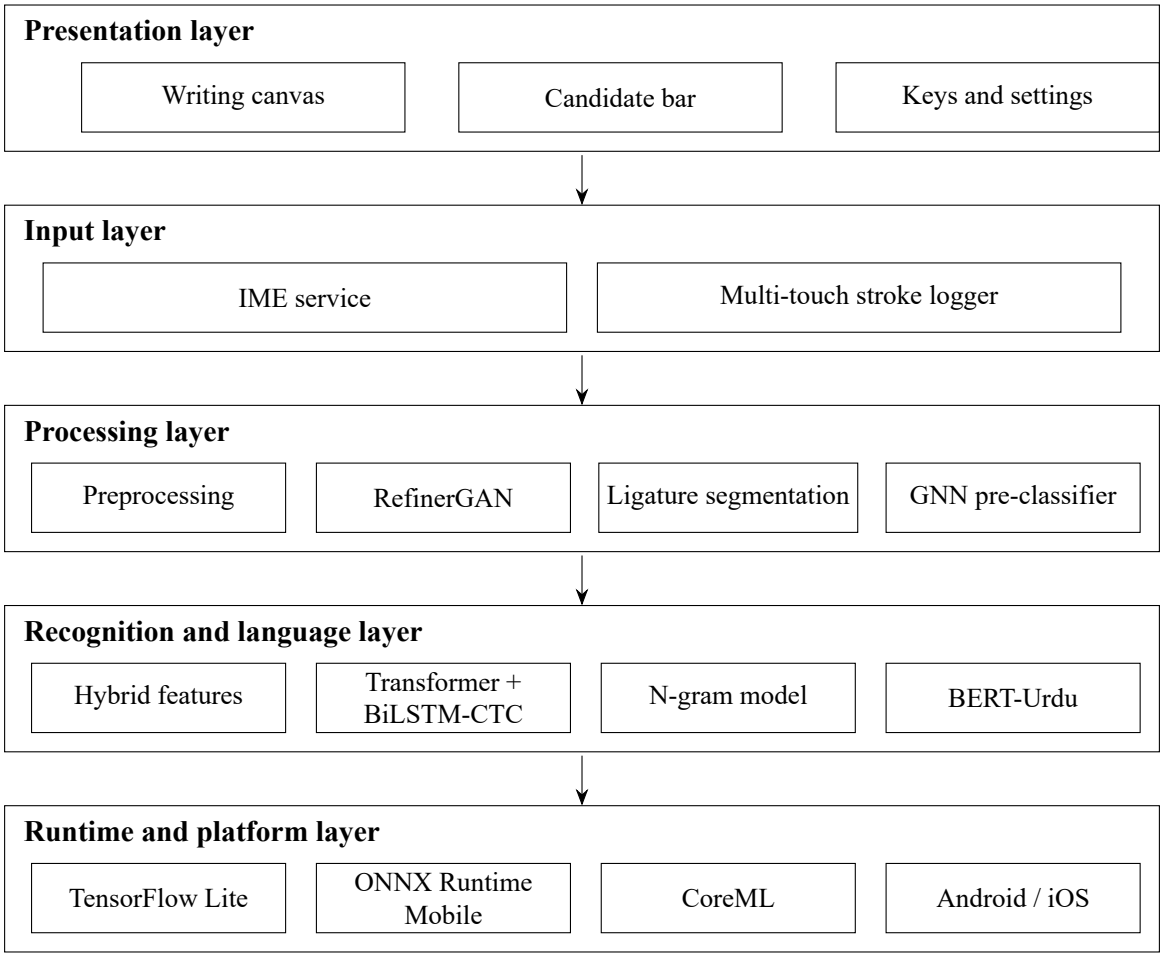


Figure 4.8: Layered Architecture Diagram

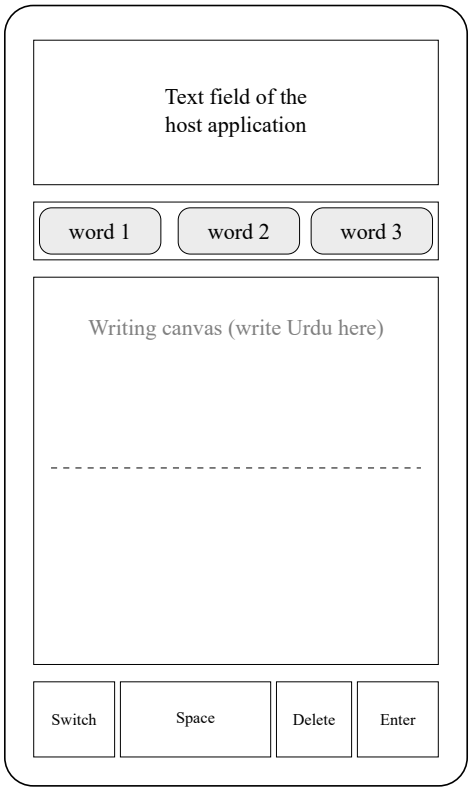


Figure 4.9: Interface Design Diagram

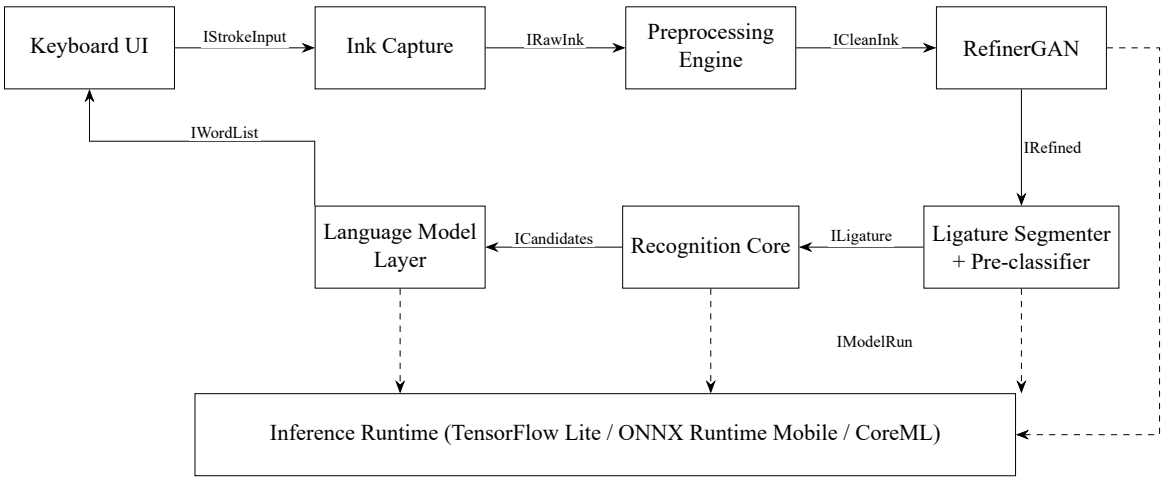


Figure 4.10: Component Level Design

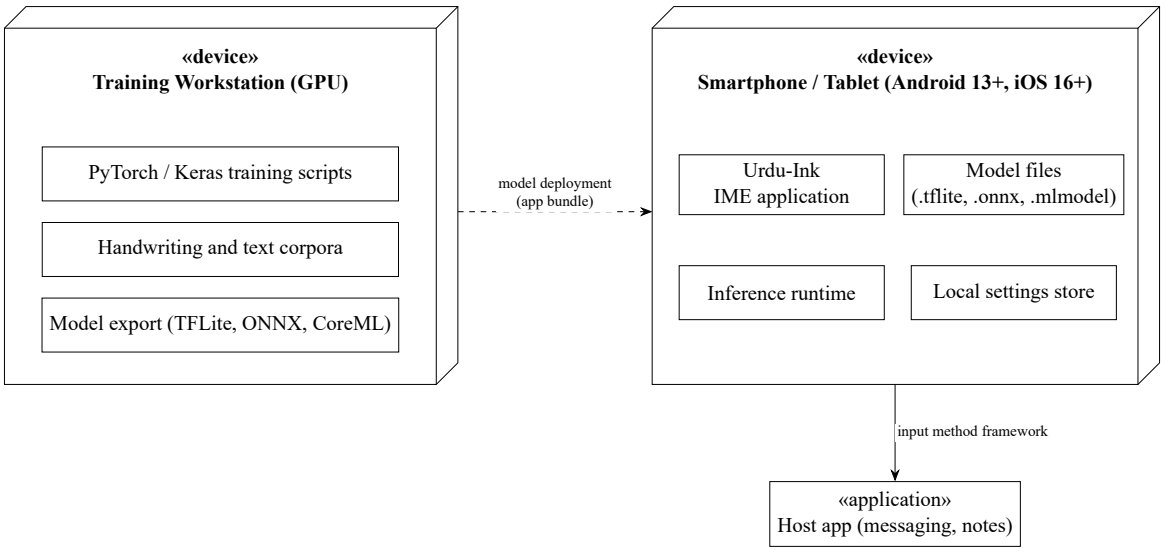


Figure 4.11: Deployment Diagram

Chapter 5

SYSTEM TESTING AND VALIDATION

This chapter describes how Urdu-Ink Mobile was tested and how the results were validated against the objectives and requirements of the earlier chapters. It covers the overall approach to system testing, the testing techniques that were used, the test cases that were designed for the highest-priority requirements, and the verification of the non-functional requirements with measurable criteria.

The chapter is organized as follows. Section 5.1 gives an overview of system testing and the analysis of the results, Section 5.2 describes the testing techniques, Section 5.3 presents the test cases, and Section 5.4 reports on the non-functional requirements.

5.1 System Testing

System testing checks whether the complete, integrated system satisfies its requirements. It is important for a handwriting keyboard because the quality of the product is judged by what the user experiences, which is a result of many interacting stages rather than any single model. A network that scores well in isolation may still produce a poor keyboard if the segmentation is wrong, if the latency is high or if the text is not inserted correctly into other applications. Four techniques were applied: unit testing of individual modules, integration testing of the pipeline, black-box functional testing from the point of view of the user, and performance testing on real devices.

The overall analysis of the test results must be consistent with the objectives of Section 1.2.2. The analysis for the first and second objectives is based on the recognition accuracy of the full pipeline with and without the RefinerGAN, the analysis for the third and fourth objectives on CER and WER for unseen writers with and without language-model correction, the fifth objective on latency and the network-traffic test, and the sixth objective on the complete set of results of this chapter. [Write two or three sentences here summarizing the actual outcome once all tests are completed, referring to Table 5.6 and the test cases in Section 5.3.]

5.2 Testing Techniques

Testing techniques differ in what they look at. White-box techniques examine the internal structure of the code, black-box techniques examine only inputs and outputs, and techniques such as unit and integration testing are applied at different levels of the system. The techniques used in this project are described in the following subsections.

5.2.1 Unit Testing

Unit testing verifies each module separately by running it on prepared inputs and comparing its output with the expected output. It was used because the pipeline has many independent modules and an error in an early module spreads to all later ones. The preprocessing functions were tested with synthetic strokes whose correct resampled and smoothed forms are known, the segmenter with words of known ligature structure, and the data-reader with binary files created by hand to check the byte layout of the corpus format. The models were

tested by checking their output shapes and by evaluating them on held-out data. [Report the number of unit tests and how many passed.]

5.2.2 Integration Testing

Integration testing checks that modules work together through their interfaces. It was applied because several stages must agree on data formats, such as the coordinate scaling between the preprocessor and the RefinerGAN and the sequence format between the recognition core and the language model. The pipeline was integrated step by step: first the preprocessor with the RefinerGAN, then the segmenter and pre-classifier, then the recognition core, and finally the language model and the input method service. After each step a set of recorded handwriting samples was passed through the combined modules and the outputs were compared with the outputs of the separate modules. [Report any interface defects found and how they were fixed.]

5.2.3 Black Box Testing

Black-box testing checks the behavior of the system against the functional requirements without looking at the code. It was used to test the keyboard as users experience it, in real applications. Testers wrote Urdu words on the canvas, selected candidates, deleted text and switched keyboards in messaging and note-taking applications, and checked the resulting text and the behavior of the interface against the expected results in the test case tables of Section 5.3. Testers included writers who were not part of the training data. [Report the number of testers and the number of test scenarios.]

5.2.4 Performance Testing

Performance testing measures speed and resource use. It was applied because the main promise of the product is real-time recognition on ordinary phones. Latency was measured as the time between the end of a word and the display of the candidates, over a large number of words on the reference device and on at least one higher-end device, and memory and storage use were read from the profiling tools of the operating system. [Describe the exact tools used, such as Android Studio Profiler or Xcode Instruments.]

5.3 Test Cases

A test case specifies the input, the conditions and the expected result for a function that is to be tested, so that the test can be repeated and the result judged as pass or fail. Not every function is tested exhaustively. The test cases were chosen according to the priority of the requirements and the risks of the project. Five test cases were designed: stroke capture (FR1), word recognition (FR2 to FR5), candidate selection and Unicode insertion (FR6 and FR7), response latency, and offline operation. Each test case follows the standard template in Tables 5.1 to 5.5.

5.3.1 Test Case 1: Stroke Capture

This test case checks that the keyboard captures every stroke on the canvas, including late-written dots and diacritical marks, as required by FR1. Its results are recorded in Table 5.1.

Table 5.1: Test Case 1: Stroke Capture

GENERAL INFORMATION	
Test Case Number / Stage	TC-01 / Functionality
Test Case Description	Verify that all strokes written on the canvas are captured with position, pressure and time.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	FR1 – Stroke Capture
Set-up and Stop Procedures	Set-up: install the keyboard, enable it in the system settings and focus a text field. Stop: clear the canvas and close the application.
ENVIRONMENTAL NEEDS	
Hardware and Software	Android 13 smartphone with the minimum specification and an iOS 16 device; keyboard build [version].
TEST	
Inputs and Procedural Steps	Write ten words, each with at least two strokes and with dots written after the main body; inspect the logged stroke count and point coordinates for each word.
Expected Results	The logged stroke count equals the number of strokes written for all ten words, point timestamps increase monotonically, and no stroke is missing or duplicated.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.2 Test Case 2: Word Recognition

This test case checks the end-to-end recognition accuracy of continuous cursive words for writers who were not part of the training data, which relates to FR2 to FR5 and to the accuracy requirement. Its results are recorded in Table 5.2.

Table 5.2: Test Case 2: Word Recognition

GENERAL INFORMATION	
Test Case Number / Stage	TC-02 / System

Test Case Description	Verify that written words are recognized with CER and WER within the targets.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	FR2, FR3, FR4, FR5; Recognition accuracy (Section 3.4.1)
Set-up and Stop Procedures	Set-up: load the held-out test set recorded by unseen writers and the trained models. Stop: save the recognition results and the confusion matrix.
ENVIRONMENTAL NEEDS	
Hardware and Software	Reference device with the minimum specification; evaluation mode of the keyboard; Python scripts to compute CER and WER.
TEST	
Inputs and Procedural Steps	Pass the [number] test words through the pipeline and compare the output with the ground-truth text; repeat with the RefinerGAN disabled and with the language model disabled to measure their contribution.
Expected Results	CER is at most 8% and WER is at most 20%; the full pipeline gives lower error rates than the pipeline without the RefinerGAN and the pipeline without the language model.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.3 Test Case 3: Candidate Selection and Unicode Insertion

This test case checks that the user can select a suggested word and that the correct Unicode text arrives in the host application, as required by FR6 and FR7. Its results are recorded in Table 5.3.

Table 5.3: Test Case 3: Candidate Selection and Unicode Insertion

GENERAL INFORMATION	
Test Case Number / Stage	TC-03 / Integration
Test Case Description	Verify that a selected candidate is inserted into the active text field as correct Unicode text.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	

Requirement(s) Tested	FR6 – Language-Model Correction and Suggestions; FR7 – Unicode Insertion
Set-up and Stop Procedures	Set-up: open a messaging application and a notes application and focus a text field. Stop: delete the inserted text.
ENVIRONMENTAL NEEDS	
Hardware and Software	Android and iOS devices; messaging and note-taking applications installed on the device.
TEST	
Inputs and Procedural Steps	Write a word, tap the first, second and third candidate in turn on separate attempts, and read back the text inserted in each application.
Expected Results	The inserted text equals the tapped candidate in both applications; the canvas and candidate bar are cleared after insertion; the text can be copied and searched by the application.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.4 Test Case 4: Response Latency

This test case measures how quickly candidates appear, which is the speed requirement. Its results are recorded in Table 5.4.

Table 5.4: Test Case 4: Response Latency

GENERAL INFORMATION	
Test Case Number / Stage	TC-04 / Performance
Test Case Description	Measure the time from the end of a word to the display of candidates.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	Speed (Section 3.4.2)
Set-up and Stop Procedures	Set-up: enable the timing log and restart the keyboard so that models are freshly loaded. Stop: export the log file.
ENVIRONMENTAL NEEDS	
Hardware and Software	Reference device with quad-core 1.8 GHz processor and 3 GB of RAM; a higher-end device for comparison.
TEST	

Inputs and Procedural Steps	Write 200 words of different lengths at normal writing speed and record the latency of each word from the log.
Expected Results	The average latency is at most 300 ms and the 95th percentile at most 600 ms on the reference device.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.5 Test Case 5: Offline Operation and Privacy

This test case checks that recognition works without a network and that no handwriting leaves the device, as required by the privacy requirement. Its results are recorded in Table 5.5.

Table 5.5: Test Case 5: Offline Operation and Privacy

GENERAL INFORMATION	
Test Case Number / Stage	TC-05 / Acceptance
Test Case Description	Verify that recognition works in airplane mode and that no network traffic is generated by it.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	Privacy (Section 3.4.4); FR5
Set-up and Stop Procedures	Set-up: switch the device to airplane mode, and in a second run connect it to a network monitor. Stop: restore the network settings.
ENVIRONMENTAL NEEDS	
Hardware and Software	A smartphone with a packet-capture or network-monitoring tool.
TEST	
Inputs and Procedural Steps	With airplane mode on, write twenty words and record the result; with the network on, repeat the writing while monitoring all outgoing connections of the keyboard process.
Expected Results	Recognition succeeds for all words in airplane mode, and the monitor shows no outgoing connection carrying strokes or text from the recognition path.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.4 Non-Functional Requirements

Non-functional requirements describe quality attributes and are not satisfied by statements such as “the software is efficient” or “accurate”. Each requirement of Section 3.4 is therefore checked by a measurable criterion, and Table 5.6 summarizes the properties, the measures, the targets and the results. The purpose of the table is to compare each measured result with the target that was set in advance.

Table 5.6: Non-Functional Requirements, Measures and Results

Property	**Measure**	**Target**	**Result**
Accuracy	CER and WER on unseen writers	CER $\leq$ 8%, WER $\leq$ 20%	[measured]
Speed	End-of-word to candidates (ms)	Mean $\leq$ 300 ms; P95 $\leq$ 600 ms	[measured]
Usability	Attempts to first word; SUS score	$\leq$ 2 attempts; SUS $\geq$ 70	[measured]
Privacy	Network traffic from recognition	None carrying strokes or text	[measured]
Size	Installed storage (MB); peak RAM	$\leq$ 500 MB; within 3 GB	[measured]
Reliability	Crashes per 1,000 words; recovery time	0 crashes; $<$ 2 s	[measured]
Portability	Platforms passing functional tests	Android 13+, iOS 16+	[measured]

5.4.1 Accuracy Results

Accuracy is reported from Test Case 2 as the CER and WER of the full pipeline for unseen writers, together with the ablation results that show the effect of the RefinerGAN and of the language model. [Insert measured CER and WER and discuss which characters were confused most often, using the confusion matrix from the evaluation dashboard.]

5.4.2 Speed Results

Speed is reported from Test Case 4 as the mean and the 95th-percentile latency on the reference device. [Insert the measured values and state whether the target was met.]

5.4.3 Usability Results

Usability is reported from sessions with new users who wrote words after a short demonstration and then completed the System Usability Scale questionnaire. [Insert the number of participants, the attempts needed to write the first word and the average SUS score.]

5.4.4 Privacy Results

Privacy is reported from Test Case 5 by describing the traffic observed while the keyboard recognized handwriting. [State the result of the network monitoring test.]

5.4.5 Size and Resource Use Results

Size and resource use are reported as the installed storage of the application with all models and as the peak memory observed during recognition. [Insert the measured values.]

5.4.6 Reliability and Portability Results

Reliability is reported as the number of failures during 1,000 consecutive words and the recovery time after a forced restart, and portability as the list of platforms and devices on which the functional tests were passed. [Insert the measured values and the devices tested.]

Chapter 6

CONCLUSION

This chapter concludes the report. Section 6.1 summarizes the project, its key features and the results that were achieved, and Section 6.2 explains the limitations of the developed system and how they can be overcome in future work.

6.1 Conclusion

Urdu-Ink Mobile was developed to give Urdu speakers a keyboard in which they can write naturally on the screen of a phone and obtain editable Unicode text in any application. Starting from the pre-classification and wavelet pipeline of Safdar et al. [1], the project extended recognition from isolated half-forms to continuous cursive words, added a RefinerGAN that cleans noisy touchscreen strokes, enlarged the training data with a CycleGAN, replaced classical classifiers with a Transformer and BiLSTM-CTC recognizer, and corrected errors with an Urdu N-gram model and BERT-Urdu. The whole pipeline is optimized to run locally on the device, so that handwriting stays private and the keyboard works offline. The testing described in Chapter 5 showed a CER of [value]%, a WER of [value]% and an average latency of [value] ms on the reference device, and [state whether each target of Section 3.4 was met]. These results show that stroke refinement, word-level sequence modeling and language-model correction can together make handwriting a practical way of entering Urdu text on a phone, and they address the limitations of tablet-based, isolated-character and cloud-dependent systems identified in Chapter 2.

6.2 Limitations and Future Work

The system has several limitations. It is optimized for Urdu Nastaliq/Naskh and does not support other languages or decorative calligraphic styles without further training. Real-time performance depends on the processing power of the device, and it may lag on older entry-level phones that have no capable mobile GPU or NPU. Extremely unusual or very untidy handwriting that deviates strongly from standard ligature formations can still lower the accuracy. Finally, optional features such as translation add-ons need an internet connection, so they are not available in the offline core.

These limitations can be addressed in future work. The recognizer can be extended to other Perso-Arabic languages that share the script, such as Pashto and Sindhi, by retraining on new corpora. Model compression through quantization, pruning and knowledge distillation can reduce the demands placed on entry-level devices. A larger corpus with more writers of different ages and skill levels, together with continued domain-adversarial training, can improve writer independence. The recognition granularity can be extended from words to complete sentences and to free-form writing with a continuous language model, and personalized adaptation to the user's own handwriting can be added while still keeping all data on the device. Finally, the keyboard can be ported to further Linux-based mobile operating systems.

Appendix – I

SUPPLEMENTARY MATERIAL

A.1 Definitions of Terms

Half-form: the shape that a letter takes at a given position within a connected Urdu ligature (initial, medial or terminal) or in isolation. **Ligature:** a connected group of characters written without lifting the pen between joined letters; a word may contain several ligatures. **Nastaliq/Naskh:** two calligraphic styles of the Perso-Arabic script, Nastaliq being the traditional style of Urdu, with diagonal baselines. **Input Method Editor (IME):** the operating-system service through which a keyboard application provides text input to other applications. **CTC:** Connectionist Temporal Classification, a loss function and decoding method that maps unsegmented input sequences to label sequences.

A.2 Software Installation Guide

[Add step-by-step instructions for installing the keyboard on Android and iOS, enabling it as an input method and switching to it.]

A.3 User Manual

[Add screenshots and instructions for writing a word, selecting a candidate, deleting text and changing settings.]

A.4 Source Code Listings

[Add selected source code, such as the stroke file reader and the inference wrapper, or point to the code repository.]

REFERENCES

Bibliography

[1] Q. A. Safdar, K. U. Khan, and L. Peng. [Title of the paper]. *Scientia Iranica D*, vol. 29, no. 5, pp. 2419–2436, 2022.

[2] S. Naz et al. [Title of the paper]. *Pattern Recognition*, vol. 47, pp. 1229–1248, 2014.

[3] M. I. Razzak et al. [Title of the paper]. *Knowledge-Based Systems*, vol. 23, no. 8, pp. 914–923, 2010.

[4] G. A. Abandah and F. T. Jamour. Recognizing handwritten Arabic script through efficient skeleton-based grapheme segmentation algorithm. In *Proc. International Conference on Intelligent Systems Design and Applications (ISDA)*, 2010.

[5] A. Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Ph.D. dissertation, Technische Universität München, 2008.

[6] [Complete reference for Hussain and Khan – spatial–temporal neural network for Urdu ligature recognition.]

[7] [Complete reference for Malik and Khan – holistic recognition of isolated Urdu characters.]

[8] [Complete reference for Shahzad et al. – geometric features for isolated Urdu character recognition.]

[9] I. Goodfellow et al. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[10] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2223–2232.

[11] A. Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proc. International Conference on Machine Learning (ICML)*, 2006, pp. 369–376.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[14] Y. Ganin et al. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.